

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное
учреждение высшего образования**

**«СИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ И ИНФОРМАТИКИ»**

(СибГУТИ)

**Кафедра телекоммуникационных систем и вычислительных средств (ТС и
ВС)**

РАСЧЁТНО-ГРАФИЧЕСКАЯ РАБОТА

**по дисциплине «Визуальное программирование и
человеко-машинное взаимодействие»**

**на тему: «ПРОГРАММНО-АППАРАТНЫЙ КОМПЛЕКС ДЛЯ
МОНИТОРИНГА И ВИЗУАЛИЗАЦИИ ПАРАМЕТРОВ
РАДИОСИГНАЛА»**

**Студент: Г. Г. Таубин
Группа: ИКС-431**

Преподаватель: Р. В. Ахпашев

Новосибирск 2026 г.

Оглавление

ВВЕДЕНИЕ	3
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
1.1 Система координат и картография	5
1.1.1 Геодезические координаты LLA	5
1.1.2 Проекция Меркатора	5
1.2 Метрики качества сотового сигнала	6
1.2.1 Общая характеристика	6
1.2.2 RSSI (Received Signal Strength Indicator)	6
1.2.3 RSRP (Reference Signal Received Power)	6
1.2.4 RSRQ (Reference Signal Received Quality)	7
1.2.5 SINR (Signal-to-Interference-plus-Noise Ratio)	7
2 ЛАБОРАТОРНЫЕ РАБОТЫ И МЕТОДЫ ИХ РЕШЕНИЯ	8
2.1 Разработка высокопроизводительного Backend-сервера на C++	8
2.1.1 Цель работы	8
2.1.2 Архитектура и стек технологий	8
2.1.3 Многопоточность и синхронизация	8
2.1.4 Обработка и парсинг данных	9
2.1.5 Графический интерфейс (ImGui & ImPlot)	11
2.2 Работа с сокетами (ZeroMQ). Передача данных от Android к PC	12
2.2.1 Цель работы	12
2.2.2 Обоснование выбора ZeroMQ	12
2.2.3 Ход выполнения	12
2.2.4 Обработка и парсинг данных	13
2.3 Взаимодействие с базой данных PostgreSQL	15
2.3.1 Цель работы	15
2.3.2 Контейнеризация и развертывание БД	15
2.3.3 Проектирование схемы данных	16
2.4 Отрисовка OpenStreetMap карты	18
2.4.1 Цель работы	18
2.4.2 Математический аппарат (Проекция Меркатора)	18
2.4.3 Асинхронный механизм загрузки тайлов	18
2.4.4 Локальное кеширование	18
2.4.5 Управление видеопамью и текстурами	19
2.5 Генерация тепловой карты (Heatmap) методом IDW	21
2.5.1 Цель работы	21
2.5.2 Метод IDW (Inverse Distance Weighting)	21
2.5.3 Вычисление тепловой карты	21
2.5.4 Цветовая шкала	22

2.6	Архитектура ПАК и результаты тестирования	24
2.6.1	Общая архитектура системы	24
2.6.2	Схема архитектуры	24
2.6.3	Структура данных	24
2.6.4	Результаты полевых испытаний	25
ЗАКЛЮЧЕНИЕ		26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		28

ВВЕДЕНИЕ

В рамках учебного курса за текущий год был реализован комплексный проект, охватывающий ключевые аспекты современной разработки инфокоммуникационных систем: от низкоуровневой оптимизации и сбора телеметрии до построения облачной серверной архитектуры и разработки мобильных интерфейсов. Данный проект представляет собой полноценный программно-аппаратный комплекс (ПАК), предназначенный для профессионального мониторинга параметров сотовой сети и анализа качества покрытия.

Объектом данной работы является программно-аппаратный комплекс для мониторинга и визуализации геоданных и параметров качества радиосигнала. В ходе выполнения практических работ был пройден полный цикл разработки системы, начиная от сбора данных на мобильном устройстве и заканчивая их визуализацией на профессиональном уровне.

Актуальность работы

С развитием технологий мобильной связи и повсеместным распространением смартфонов, вопрос качества покрытия сотовых сетей становится все более актуальным. Операторы связи нуждаются в эффективных инструментах для мониторинга параметров сети, выявления "мертвых зон" и оптимизации расположения базовых станций. Разработанный в рамках данного проекта ПАК позволяет решать эти задачи с минимальными затратами, используя обычный смартфон в качестве измерительного прибора.

Цели и задачи проекта

Основной целью проекта является создание полнофункциональной системы для сбора, хранения, анализа и визуализации параметров мобильной сети. Для достижения этой цели были решены следующие задачи:

- Разработка мобильного приложения для сбора телеметрии в реальном времени;
- Создание высокопроизводительного сервера на C++ для обработки входящих данных;
- Интеграция с системой управления базами данных PostgreSQL для долгосрочного хранения информации;
- Реализация графического интерфейса для визуализации данных на картах OpenStreetMap;
- Разработка алгоритмов построения тепловых карт для анализа покрытия сети.

Структура отчета

Данный отчет состоит из трех основных разделов. В первом разделе рассматриваются теоретические основы работы с геоданными и параметрами радиосигнала. Во втором разделе подробно описываются все этапы практической реализации проекта, включая разработку серверной части, работу с базами данных, отрисовку карт и генерацию тепловых карт. В третьем разделе представлены результаты тестирования системы в реальных условиях и выводы по проделанной работе.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Система координат и картография

1.1.1 Геодезические координаты LLA

Для определения местоположения объектов в пространстве в рамках данного проекта используется система координат LLA (Latitude, Longitude, Altitude). Эта система базируется на геодезической модели Земли, где планета представляется в виде эллипсоида вращения. Основными параметрами в этой системе являются:

- **Широта (Latitude)** — угол между плоскостью экватора и нормалью к поверхности эллипсоида в данной точке. Измеряется от -90° (Южный полюс) до $+90^\circ$ (Северный полюс).
- **Долгота (Longitude)** — угол между плоскостью нулевого (Гринвичского) меридиана и плоскостью меридиана, проходящего через точку. Измеряется от -180° до $+180^\circ$.
- **Высота (Altitude)** — расстояние от поверхности эллипсоида до точки по нормали. В задачах телеметрии этот параметр важен для учета рельефа местности и высоты подвеса антенн.

Точность определения координат с помощью GPS-модуля современного смартфона может достигать 3-5 метров в открытой местности, что вполне достаточно для задач мониторинга покрытия сотовой сети.

1.1.2 Проекция Меркатора

Для отображения сферических координат Земли на плоском экране компьютера необходима математическая проекция. В реализованном программном обеспечении координаты LLA преобразуются в пиксельные координаты экранного пространства с использованием проекции Меркатора. Эта проекция была выбрана по нескольким причинам:

1. Она сохраняет углы (является конформной), что важно для навигационных задач;
2. Большинство онлайн-карт (OpenStreetMap, Google Maps, Яндекс.Карты) используют именно эту проекцию;
3. Существуют простые математические формулы для прямого и обратного преобразования координат.

Формулы преобразования выглядят следующим образом:

$$x = R \cdot \lambda$$

$$y = R \cdot \ln \left(\tan \left(\frac{\pi}{4} + \frac{\varphi}{2} \right) \right)$$

где R — радиус Земли, λ — долгота в радианах, φ — широта в радианах.

В рамках проекта были реализованы функции `TileXExact` и `TileYExact`, которые вычисляют позицию тайла на основе широты, долготы и уровня масштабирования, что позволяет динамически подгружать необходимые фрагменты карты.

1.2 Метрики качества сотового сигнала

1.2.1 Общая характеристика

Для оценки состояния канала связи в мобильных сетях используются различные метрики, которые позволяют количественно оценить качество соединения. В рамках данного проекта с мобильного устройства считываются и передаются на сервер следующие параметры:

1.2.2 RSSI (Received Signal Strength Indicator)

RSSI представляет собой общий индикатор мощности принимаемого сигнала. Этот параметр измеряется в дБм (dBm) и рассчитывается по формуле:

$$RSSI = 10 \cdot \log_{10} \left(\frac{P}{1\text{мВт}} \right)$$

Важно отметить, что RSSI учитывает всю энергию в полосе частот, включая не только полезный сигнал, но и шумы, а также интерференцию от соседних передатчиков. Типичные значения RSSI варьируются в следующих диапазонах:

- **-50 дБм и выше** — отличный сигнал, максимальная скорость передачи данных;
- **-60 до -70 дБм** — хороший сигнал, стабильное соединение;
- **-70 до -85 дБм** — удовлетворительный сигнал, возможны незначительные проблемы;
- **-85 до -100 дБм** — слабый сигнал, низкая скорость;
- **-100 дБм и ниже** — связь практически отсутствует.

1.2.3 RSRP (Reference Signal Received Power)

RSRP является ключевым параметром для сетей LTE (4G) и более новых стандартов. В отличие от RSSI, RSRP измеряет мощность только опорных сигналов (Reference Signals), которые базовые станции передают на определенных ресурсных элементах. Это позволяет более точно оценить уровень покрытия конкретной базовой станции без учета шумов в канале и интерференции.

Для RSRP также существуют характерные пороговые значения:

- > -80 дБм — отличный сигнал, максимальная производительность;
- -80 до -95 дБм — хороший сигнал;
- -95 до -105 дБм — средний сигнал;
- -105 до -115 дБм — слабый сигнал, возможны обрывы;
- < -115 дБм — очень слабый сигнал, соединение нестабильно.

В проекте значения RSRP используются для построения тепловых карт (heatmaps), где цвет каждой точки на карте соответствует уровню сигнала, что позволяет визуально оценить "мертвые зоны" покрытия.

1.2.4 RSRQ (Reference Signal Received Quality)

RSRQ представляет собой отношение RSRP к RSSI и характеризует качество канала с учетом уровня помех. Этот параметр особенно важен в условиях плотной городской застройки, где наблюдается высокий уровень интерференции от соседних базовых станций.

1.2.5 SINR (Signal-to-Interference-plus-Noise Ratio)

SINR показывает отношение мощности полезного сигнала к сумме мощностей шума и интерференции. Этот параметр наиболее полно характеризует качество канала связи и напрямую влияет на достигаемую скорость передачи данных.

2 ЛАБОРАТОРНЫЕ РАБОТЫ И МЕТОДЫ ИХ РЕШЕНИЯ

2.1 Разработка высокопроизводительного Backend-сервера на C++

2.1.1 Цель работы

Основной целью данной лабораторной работы являлась реализация серверной логики на языке программирования C++ для обеспечения максимальной производительности при обработке большого количества входящих телеметрических данных. Дополнительными задачами стали: реализация многопоточной архитектуры приложения и создание графической оболочки на базе библиотеки `ImGui` для визуализации телеметрии в реальном времени.

2.1.2 Архитектура и стек технологий

Серверная часть была полностью переписана на языке C++ с использованием стандарта C++17. В качестве системы сборки использован `CMake` версии 3.15 и выше, что позволило удобно управлять внешними зависимостями и обеспечило кроссплатформенность сборки. Для работы с различными библиотеками были использованы следующие инструменты:

- `libzmq` — библиотека для высокопроизводительного обмена сообщениями;
- `libpq` — официальный клиент для работы с PostgreSQL;
- `OpenGL` — для аппаратно-ускоренной отрисовки графики;
- `SDL2` — для создания окна и обработки событий;
- `ImGui` и `ImPlot` — для создания графического интерфейса и построения графиков.

Библиотеки `ImGui` и `ImPlot` были интегрированы в проект как `git submodules`, что обеспечивает чистоту структуры репозитория и простоту развертывания на различных операционных системах, включая Ubuntu Linux и Windows.

2.1.3 Многопоточность и синхронизация

Ключевой особенностью реализации стало разделение логики приложения на два независимых потока выполнения:

1. **Сетевой поток (`zmq_receiver`):** Этот поток работает в фоновом режиме, обеспечивая постоянное прослушивание сетевого порта на предмет входящих сообщений от мобильных клиентов. В нем реализована логика автоматического переподключения клиента и подтверждения

приема данных (отправка ответа "ОК"). При получении нового пакета данных, сетевой поток производит его первичную обработку и сохраняет в общую структуру данных.

2. **Поток визуализации (Main/GUI):** Этот поток отвечает за отрисовку графического интерфейса пользователя, обновление графиков, карт и текстовых полей. Он работает с частотой обновления экрана (обычно 60 FPS), что обеспечивает плавную анимацию и мгновенную реакцию на действия пользователя.

Связующим звеном между двумя потоками выступила глобальная структура `RuntimeState`, которая содержит все актуальные данные о состоянии системы. Для обеспечения потокобезопасности при одновременном доступе (сетевой поток записывает координаты и параметры сигнала, а GUI поток их читает для отображения) был использован `std::mutex`. Это гарантирует, что не возникнет состояния гонки (race condition) или повреждения данных. Фрагмент кода, демонстрирующий использование мьютекса, представлен в листинге 2.1.

Листинг 2.1: Использование мьютекса для синхронизации потоков

```
1 //
2 struct TelemetryData {
3     std::mutex mtx;
4     double latitude;
5     double longitude;
6     double altitude;
7     int rsrp;
8     // ...
9 };
10
11 //
12 void OnDataReceived(const json& data) {
13     std::lock_guard<std::mutex> lock(g_data.mtx);
14     g_data.latitude = data["lat"];
15     g_data.longitude = data["lon"];
16     g_data.rsrp = data["rsrp"];
17 }
18
19 // GUI
20 void DisplayData() {
21     std::lock_guard<std::mutex> lock(g_data.mtx);
22     ImGui::Text("Latitude: %.6f", g_data.latitude);
23     ImGui::Text("Longitude: %.6f", g_data.longitude);
24 }
```

2.1.4 Обработка и парсинг данных

Данные от Android-приложения поступают на сервер в формате JSON (JavaScript Object Notation). Этот формат был выбран из-за его удобочитаемости, гибко-

сти и поддержки на всех платформах. Для обработки входящих сообщений была написана функция `ApplyPacket` (файл `json_parser.cpp`), которая выполняет следующие действия:

1. Парсинг JSON-строки в объектную модель;
2. Извлечение навигационных данных (широта, долгота, высота, точность);
3. Извлечение параметров радиосигнала (RSRP, RSRQ, RSSI, PCI, EARFCN);
4. Сохранение извлеченных данных в структуру `RuntimeState`;
5. При необходимости — запись в базу данных PostgreSQL.

Помимо структурированных данных, в системе сохраняется «сырая» строка JSON, что полезно для отладки и контроля полноты передаваемой информации. Основной цикл обработки сообщений представлен в листинге 2.2.

Листинг 2.2: Основной цикл графического интерфейса (`gui_app.cpp`)

```
1 bool GuiAppFrame(RuntimeState& state) {
2     SDL_Event e;
3     while (SDL_PollEvent(&e)) {
4         ImGui_ImplSDL2_ProcessEvent(&e);
5         if (e.type == SDL_QUIT) state.running.store(false);
6     }
7
8     ImGui_ImplOpenGL3_NewFrame();
9     ImGui_ImplSDL2_NewFrame();
10    ImGui::NewFrame();
11
12    ImGui::Begin("##root", nullptr, ImGuiWindowFlags_NoDecoration);
13    ImGui::Text("                : %s |                : %d",
14                state.packets.load() > 0 ? "                " : "                ",
15                state.packets.load());
16
17    if (ImGui::BeginTabBar("tabs")) {
18        if (ImGui::BeginTabItem("                ")) { TabMap(state);
19        ImGui::EndTabItem(); }
20        if (ImGui::BeginTabItem("                ")) { TabSignals(
21        state); ImGui::EndTabItem(); }
22        if (ImGui::BeginTabItem("                ")) {
23        TabTelephony(state); ImGui::EndTabItem(); }
24        if (ImGui::BeginTabItem("                ")) { TabFilters(
25        state); ImGui::EndTabItem(); }
26        ImGui::EndTabBar();
27    }
28    ImGui::End();
29}
```

```

26     ImGui::Render();
27     glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
28     ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());
29     SDL_GL_SwapWindow(s_window);
30     return true;
31 }

```

2.1.5 Графический интерфейс (ImGui & ImPlot)

Был разработан многооконный интерфейс, включающий в себя несколько вкладок:

- **Главная вкладка** — отображает текущие данные о местоположении и параметрах сигнала;
- **Графики** — показывает временные ряды RSRP, RSRQ и RSSI для различных базовых станций;
- **Карта** — отображает OpenStreetMap карту с наложенным треком передвижения;
- **Телефония** — детальная информация о текущих и соседних сотах;
- **Фильтры** — настройка фильтрации данных для тепловых карт.

Использование библиотеки ImPlot позволило создать профессиональные графики с возможностью масштабирования, панорамирования и отображения легенды.

Итог

В результате выполнения данной лабораторной работы разработано стабильное серверное приложение на C++, обеспечивающее параллельную работу сетевого стека и графической оболочки. Система корректно принимает телеметрию, сохраняет её в памяти и отображает пользователю актуальные параметры местоположения и состояния сети в режиме реального времени.

2.2 Работа с сокетами (ZeroMQ). Передача данных от Android к PC

2.2.1 Цель работы

Целью данной работы являлось изучение высокоуровневых механизмов работы с сетевыми сокетами через библиотеку ZeroMQ, а также реализация устойчивого канала связи между мобильным устройством под управлением Android и персональным компьютером для передачи телеметрических данных в реальном времени.

2.2.2 Обоснование выбора ZeroMQ

Традиционные сокеты (Berkeley sockets) предоставляют низкоуровневый интерфейс для сетевого взаимодействия, однако требуют от разработчика реализации множества дополнительных механизмов: буферизация, переподключение при обрыве связи, маршрутизация сообщений и т.д. ZeroMQ (также известная как ØMQ, 0MQ или ZMQ) представляет собой библиотеку асинхронных сообщений, которая решает эти проблемы "из коробки". Основные преимущества ZeroMQ:

- Поддержка различных топологий взаимодействия (PUSH/PULL, PUB/SUB, REQ/REP);
- Автоматическое переподключение при обрыве соединения;
- Асинхронная буферизация сообщений;
- Кроссплатформенность (работает на Windows, Linux, Android, iOS);
- Высокая производительность (десятки тысяч сообщений в секунду).

2.2.3 Ход выполнения

В мобильном приложении была реализована новая активность, в которой тестировалась работа ZMQ-узлов внутри самого устройства. Для изоляции нового функционала и соблюдения правил командной разработки, все изменения вносились в отдельную ветку (branch) репозитория. Был реализован асинхронный запуск клиента и сервера, что позволило приложению отправлять и принимать данные «самому себе» для проверки корректности работы библиотеки в среде Android.

Для обеспечения связи между смартфоном и компьютером на домашнем маршрутизаторе был открыт определённый порт. Особое внимание уделялось определению IP-адресов узлов — на мобильном устройстве необходимо было указать конкретный сокет с IP-адресом ПК в локальной сети. Android-устройство было настроено на работу в режиме клиента, инициирующего соединение с сервером на ПК.

В функционал сервера была заложена логика накопления информации. Каждый блок данных, поступивший с мобильного клиента, фиксировался в JSON-файл для последующего анализа и отладки.

2.2.4 Обработка и парсинг данных

Данные от Android-приложения поступают на сервер в формате JSON. Для их обработки написана функция `ApplyPacket` (файл `json_parser.cpp`), которая извлекает значения координат и параметров радиосигнала. Код обработки представлен в листинге 2.3.

Листинг 2.3: Обработка входящего сообщения (`zmq_receiver.cpp`)

```
1 void RunZmqReceiver(RuntimeState* st, int dataPort, bool writeDb)
2 {
3     zmq::context_t ctx(1);
4     zmq::socket_t pull(ctx, ZMQ_PULL);
5     pull.bind("tcp://0.0.0.0:" + std::to_string(dataPort));
6     pull.set(zmq::sockopt::rcvtimeo, 100);
7
8     std::ofstream log("build/received_android.json", std::ios::
9     app);
10
11     while (st->running.load()) {
12         zmq::message_t msg;
13         if (!pull.recv(msg, zmq::recv_flags::none)) continue;
14
15         const std::string raw(static_cast<char*>(msg.data()), msg
16         .size());
17         st->bytesRx.fetch_add((long long)raw.size());
18
19         {
20             std::lock_guard<std::mutex> lk(st->mtx);
21             st->lastRawJson = raw;
22             ApplyPacket(st, raw, writeDb, db);
23         }
24
25         st->packets.fetch_add(1);
26         log << raw << '\n';
27     }
28 }
```

Результаты

В результате выполнения работы было реализовано стабильное сетевое взаимодействие между Android-устройством и C++ сервером. Система способна обрабатывать непрерывный поток телеметрических данных без потерь и задержек.

Socket Communication

Status: Ready

Message to send:

Hello from Android!

External Server IP:

192.168.1.100

My IP

Start Internal Server

Send to Internal Server

Send to External Server

Log:

[02:43:40] Application started

2.3 Взаимодействие с базой данных PostgreSQL

2.3.1 Цель работы

Основной целью данной работы являлась реализация системы долгосрочного хранения телеметрических данных с использованием промышленной СУБД PostgreSQL. Дополнительными задачами стали: настройка контейнеризированной среды базы данных с помощью Docker, проектирование оптимальной схемы данных и разработка механизмов записи данных из C++ приложения в реальном времени.

2.3.2 Контейнеризация и развертывание БД

Для обеспечения переносимости и быстрой настройки среды разработки и эксплуатации была использована технология контейнеризации Docker. С помощью файла `docker-compose.yml` был развернут контейнер с образом `postgres:16`. Преимущества такого подхода очевидны:

- База данных разворачивается одной командой `docker-compose up -d`;
- Исключаются проблемы с зависимостями и версиями на разных компьютерах;
- Легко создавать резервные копии и мигрировать между серверами;
- Возможность быстрого сброса состояния базы данных для тестирования.

Пример файла `docker-compose.yml` для развертывания PostgreSQL:

Листинг 2.4: `docker-compose.yml` для PostgreSQL

```
1 version: '3.8'
2 services:
3   postgres:
4     image: postgres:16-alpine
5     container_name: telemetry_db
6     environment:
7       POSTGRES_DB: telemetry
8       POSTGRES_USER: admin
9       POSTGRES_PASSWORD: secret_password
10    ports:
11      - "5432:5432"
12    volumes:
13      - postgres_data:/var/lib/postgresql/data
14    restart: unless-stopped
15
16 volumes:
17   postgres_data:
```

2.3.3 Проектирование схемы данных

В базе данных была создана таблица `user_equipment`, поля которой строго соответствуют собираемой телеметрии. Особое внимание было уделено выбору типов данных и созданию индексов для ускорения запросов. Структура таблицы включает следующие поля:

- **Навигационные данные:** широта (DOUBLE PRECISION), долгота (DOUBLE PRECISION), высота (DOUBLE PRECISION), точность (DOUBLE PRECISION);
- **Временная метка:** timestamp (BIGINT) для построения временных рядов;
- **Идентификаторы сети:** IMEI устройства, тип сети (TEXT), MCC, MNC, Cell ID, TAC;
- **Радиочастотные метрики:** PCI, RSRP, RSRQ, RSSI, SINR, EARFCN, NRARFCN;

Для оптимизации производительности запросов был создан составной индекс по полям (`ts`, `pci`). Фрагмент кода для создания таблицы и вставки данных представлен в листинге 2.5.

Листинг 2.5: Работа с PostgreSQL (pgsql_minimal.cpp)

```
1 bool PgSqlMinimal::ensureTable() {
2     const char* q = "CREATE TABLE IF NOT EXISTS user_equipment ("
3                     "id SERIAL PRIMARY KEY,"
4                     "imei TEXT, lat DOUBLE PRECISION, lon DOUBLE
5                     PRECISION, "
6                     "alt DOUBLE PRECISION, accuracy DOUBLE
7                     PRECISION, "
8                     "ts BIGINT, network_type TEXT, mcc TEXT, mnc
9                     TEXT, "
10                    "cell_id TEXT, tac TEXT, pci INT, rsrp INT,
11                    rsrq INT, "
12                    "rssi INT, earfcn TEXT"
13                    ")";
14    PGresult* r = PQexec(conn_, q);
15    bool ok = r && PQresultStatus(r) == PGRES_COMMAND_OK;
16    PQclear(r);
17    return ok;
18 }
19
20 bool PgSqlMinimal::insertRow(const DbSignalRow& x) {
21     const char* vals[] = {x.imei.c_str(), x.lat.c_str(), x.lon.
22                           c_str(),
23                           x.alt.c_str(), x.accuracy.c_str(), x.ts
24                           .c_str(),
25                           x.network_type.c_str(), x.pci.c_str(),
```

```

20         x.rsrp.c_str(), x.rsrq.c_str(), x.rssi.
    c_str());
21
22     const char* q = "INSERT INTO user_equipment(imei,lat,lon,alt,
    accuracy,"
23                     "ts,network_type,pci,rsrp,rsrq,rssi)"
24                     "VALUES($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11)";
25
26     PGresult* r = PQexecParams(conn_, q, 11, nullptr, vals,
    nullptr, nullptr, 0);
27     bool ok = r && PQresultStatus(r) == PGRES_COMMAND_OK;
28     PQclear(r);
29     return ok;
30 }

```

Результаты

В итоге Backend-сервер превратился в полноценную аналитическую платформу. Система успешно аккумулирует данные в СУБД PostgreSQL, обеспечивая сохранность результатов драйв-тестов, и одновременно предоставляет наглядный графический интерфейс для мониторинга с ключевыми характеристиками радиосигнала.

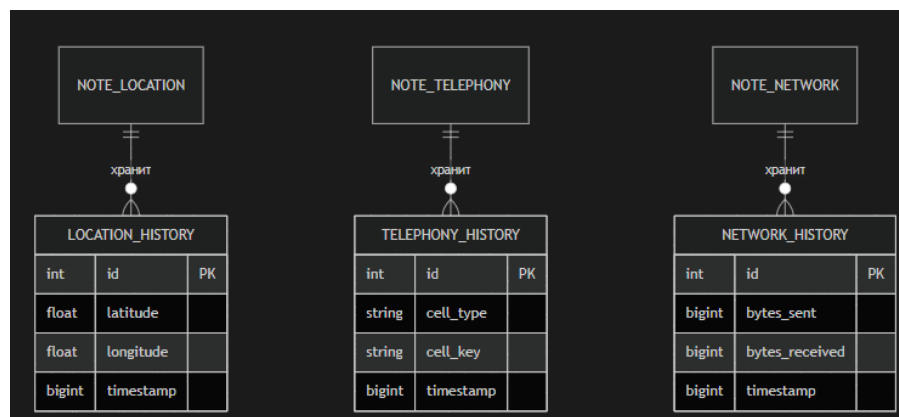


Рис. 2.2: Схема таблиц в базе данных серверного приложения

2.4 Отрисовка OpenStreetMap карты

2.4.1 Цель работы

Разработка модуля интерактивной карты, реализующего динамическую загрузку тайлов OSM (OpenStreetMap), преобразование географических координат в экранные с использованием проекции Меркатора и наложение телеметрического трека на картографическую подложку.

2.4.2 Математический аппарат (Проекция Меркатора)

Для отображения сферических координат Земли на плоскости экрана были реализованы функции преобразования `TileXExact` и `TileYExact`. Система поддерживает динамический расчет уровня масштабирования (`zoom`) в зависимости от текущих границ окна `ImPlot`, что позволяет плавно переходить от обзора города к детализации отдельных улиц.

Функции преобразования координат в номера тайлов:

Листинг 2.6: Преобразование координат в тайлы (`osm_math.h`)

```
1 // X-
2 inline double TileXExact(double lon, int zoom) {
3     return (lon + 180.0) / 360.0 * (1 << zoom);
4 }
5
6 // Y-
7 inline double TileYExact(double lat, int zoom) {
8     double lat_rad = lat * M_PI / 180.0;
9     return (1.0 - asinh(tan(lat_rad)) / M_PI) / 2.0 * (1 << zoom)
10 ;
11 }
```

2.4.3 Асинхронный механизм загрузки тайлов

Для загрузки тайлов через HTTP используется библиотека `libcurl`. Чтобы графический интерфейс не «замораживался» во время ожидания ответа от сервера OSM, загрузка каждого тайла вынесена в отдельный поток (`std::thread`) или пул потоков (`ThreadPool`).

2.4.4 Локальное кеширование

Реализована иерархическая система хранения данных в директории `tiles/{z}/{x}/`. Перед запросом в сеть сервер проверяет наличие файла на диске, что радикально снижает нагрузку на канал связи и ускоряет рендеринг. Кеширование особенно важно при многократном использовании приложения — однажды загруженные тайлы сохраняются и не требуют повторной загрузки.

2.4.5 Управление видеопамью и текстурами

С помощью библиотеки `stb_image` загруженные PNG-файлы декодируются в RAW-массивы пикселей (формат RGBA) и передаются в видеопамью как текстуры OpenGL (тип `GLuint`). Для оптимизации ресурсов реализован менеджер кеша текстур, который предотвращает утечки памяти при длительной работе программы и автоматически выгружает текстуры, которые давно не использовались. Код загрузки тайлов представлен в листинге 2.7.

Листинг 2.7: Загрузка и сохранение тайла (`osm_tile_fetcher.cpp`)

```
1 void OsmTileFetcher::ProcessJob(const Job& job) {
2     std::filesystem::path dir = std::filesystem::path("build") /
3                               std::to_string(job.coord.zoom) /
4                               std::to_string(job.coord.x);
5     std::filesystem::path filePath = dir / (std::to_string(job.
6     coord.y) + ".png");
7
8     //
9
10    if (std::filesystem::exists(filePath)) {
11        std::ifstream file(filePath, std::ios::binary | std::ios
12        ::ate);
13        if (file.is_open()) {
14            auto size = file.tellg();
15            file.seekg(0, std::ios::beg);
16            std::vector<std::byte> blob((size_t)size);
17            file.read(reinterpret_cast<char*>(blob.data()), size)
18        ;
19            job.callback(job.coord, blob);
20            return;
21        }
22    }
23
24    //
25    CURL* curl = curl_easy_init();
26    std::string url = "https://tile.openstreetmap.org/" +
27                    std::to_string(job.coord.zoom) + '/' +
28                    std::to_string(job.coord.x) + '/' +
29                    std::to_string(job.coord.y) + ".png";
30
31    curl_easy_setopt(curl, CURLOPT_URL, url.c_str());
32    curl_easy_setopt(curl, CURLOPT_USERAGENT, "TelemetryMonitor
33    /1.0");
34    curl_easy_setopt(curl, CURLOPT_TIMEOUT, 10L);
35
36    // ...
37 }
```

Результаты

Создан высокопроизводительный инструмент визуализации, способный в реальном времени отображать маршрут драйв-теста на подробной карте мира. Система автоматически подгружает необходимые фрагменты карты, сохраняет их для офлайн-использования и обеспечивает плавную навигацию с возможностью масштабирования и панорамирования.

2.5 Генерация тепловой карты (Heatmap) методом IDW

2.5.1 Цель работы

Основной целью данной работы являлась реализация алгоритма пространственной интерполяции Inverse Distance Weighting (IDW) для создания визуального слоя тепловых карт. Этот слой накладывается поверх карты OpenStreetMap и позволяет анализировать радиочастотное покрытие по различным критериям: RSRP, RSRQ, RSSI, а также визуализировать рельеф местности (Altitude).

2.5.2 Метод IDW (Inverse Distance Weighting)

Для заполнения пространства между точками измерений был реализован метод обратно взвешенных расстояний. Суть метода заключается в том, что значение в искомой точке является средневзвешенным от значений окружающих её реальных точек измерений, причем весовой коэффициент обратно пропорционален расстоянию в некоторой степени p :

$$Z(x, y) = \frac{\sum_{i=1}^n \frac{Z_i}{d_i^p}}{\sum_{i=1}^n \frac{1}{d_i^p}}$$

где:

- $Z(x, y)$ — интерполированное значение в точке (x, y) ;
- Z_i — значение в известной точке i ;
- d_i — расстояние от точки интерполяции до известной точки i ;
- p — степень, контролирующая скорость убывания веса (в проекте p настраивается от 1 до 4).

При $p = 1$ вес убывает линейно, при $p = 2$ — по квадратичному закону, что дает более гладкую интерполяцию, но и меньшее влияние дальних точек.

2.5.3 Вычисление тепловой карты

Вычисление тепловой карты — ресурсоемкая операция, требующая обхода тысяч пикселей и сопоставления их с тысячами записей в базе данных. Для каждого пикселя результирующего изображения необходимо найти все измерительные точки в радиусе влияния и вычислить средневзвешенное значение. Чтобы графический интерфейс оставался отзывчивым во время этих вычислений, математические расчеты вынесены в фоновый поток (класс `HeatmapGenerator`).

Алгоритм вычисления:

1. Определение границ области построения (bounding box);
2. Создание сетки пикселей (по умолчанию 400×400);
3. Для каждого пикселя — поиск всех точек в радиусе влияния;

4. Вычисление весов и средневзвешенного значения;
5. Преобразование полученного значения в цвет согласно выбранной палитре.

2.5.4 Цветовая шкала

Для визуализации RSRP использована классическая инженерная палитра, интуитивно понятная специалистам в области связи:

- **Красный цвет** (> -80 дБм) — отличный сигнал, максимальная мощность;
- **Желто-оранжевые тона** ($-80 \dots -95$ дБм) — хороший сигнал;
- **Зеленые тона** ($-95 \dots -105$ дБм) — средний сигнал;
- **Голубые тона** ($-105 \dots -115$ дБм) — слабый сигнал;
- **Темно-синий цвет** ($-115 \dots -120$ дБм) — очень слабый сигнал;
- **Прозрачные пиксели** (< -120 дБм) — связь практически отсутствует.

Код реализации алгоритма IDW представлен в листинге 2.8.

Листинг 2.8: Алгоритм IDW (heatmap_generator.cpp)

```
1 HeatmapCache GenerateHeatmap(const std::vector<HeatmapPoint>& pts
2                               ,
3                               const HeatmapConfig& cfg, ...) {
4     const int W = 400, H = 400;
5     std::vector<double> sumW(W * H, 0.0);
6     std::vector<double> sumV(W * H, 0.0);
7
8     //
9
10    for (const auto& pt : pts) {
11        //
12
13        for (int py = y0; py <= y1; ++py) {
14            for (int px = x0; px <= x1; ++px) {
15                double d = haversine(p_lat, p_lon, pt.lat, pt.lon);
16            };
17            if (d <= cfg.radius) {
18                double w = 1.0 / std::pow(std::max(0.1, d),
19                cfg.power);
20                sumW[py * W + px] += w;
21                sumV[py * W + px] += w * pt.value;
22            }
23        }
24    }
25 }
```

```

20     }
21
22     //
23
24     for (int i = 0; i < W * H; ++i) {
25         if (sumW[i] > 1e-12) {
26             double z = sumV[i] / sumW[i];
27             Color col = get_color(z, cfg.criterion);
28             pixels[i * 4 + 0] = col.r;
29             pixels[i * 4 + 1] = col.g;
30             pixels[i * 4 + 2] = col.b;
31             pixels[i * 4 + 3] = col.a;
32         }
33     }
34
35     return cache;
36 }

```

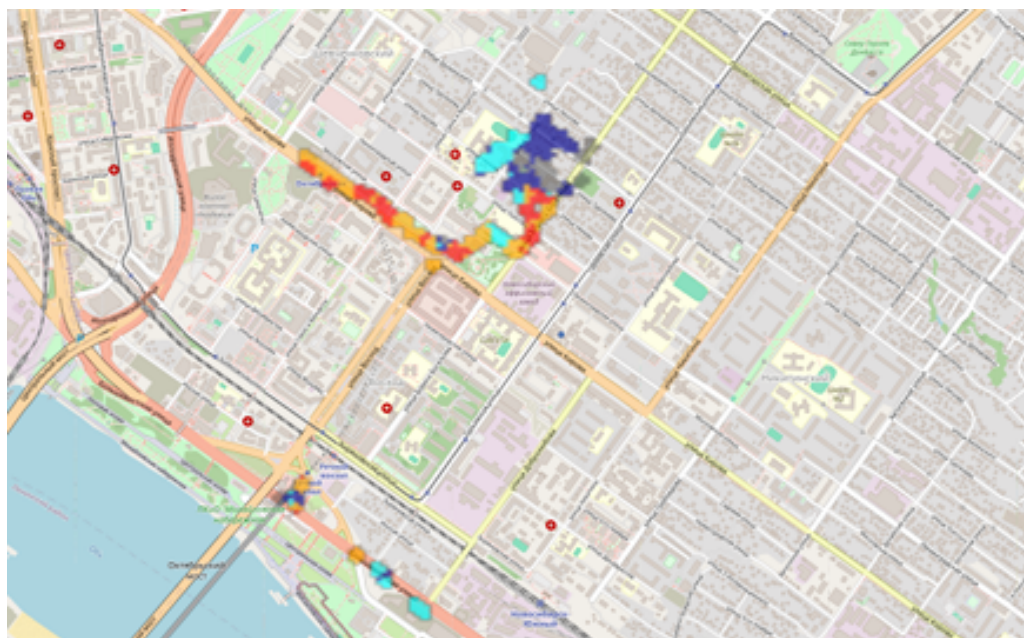


Рис. 2.3: Скриншот готового к работе backend-сервера (построена тепловая карта RSRP)

Результаты

Реализован полноценный модуль генерации тепловых карт, который позволяет анализировать покрытие сотовой сети в любой точке маршрута. Тепловая карта генерируется в виде растрового слоя, адаптированного под текущий уровень масштабирования и координаты. Полученные изображения кэшируются для ускорения повторного отображения.

2.6 Архитектура ПАК и результаты тестирования

2.6.1 Общая архитектура системы

Итоговая архитектура разработанного программно-аппаратного комплекса представляет собой распределенную систему, состоящую из следующих компонентов:

1. **Мобильное приложение (Android)** — собирает данные с GPS-модуля и модема смартфона, формирует JSON-пакеты и отправляет их на сервер через ZeroMQ.
2. **Серверное приложение (C++)** — принимает телеметрию, парсит JSON, отображает данные в реальном времени на карте и графиках.
3. **База данных (PostgreSQL)** — сохраняет все исторические данные для последующего анализа и построения тепловых карт.
4. **Тайловый сервер OSM** — предоставляет картографическую подложку через публичные API OpenStreetMap.

Поток данных в системе выглядит следующим образом:

1. Android-устройство каждые 1-2 секунды опрашивает сенсоры и модем;
2. Сформированный JSON-пакет отправляется через ZeroMQ (топология PUSH/PULL);
3. C++ сервер принимает пакет, парсит и сохраняет в PostgreSQL;
4. GUI поток обновляет отображение на карте и графиках;
5. Пользователь может в любой момент запустить генерацию тепловой карты на основе накопленных данных.

2.6.2 Схема архитектуры

2.6.3 Структура данных

В листинге 2.9 представлена основная структура данных, используемая для хранения телеметрии.

Листинг 2.9: Структура TelemetryData

```
1 struct TelemetryData {  
2     double latitude = 0;  
3     double longitude = 0;  
4     double altitude = 0;  
5     double accuracy = 0;  
6     int rsrp = 0;  
7     int rsrq = 0;  
8     int rssi = 0;
```

```

9      std::string network_type;
10     std::string cell_identity;
11     int pci = -1;
12     long long timestamp = 0;
13     std::mutex mtx;
14
15     //
16     std::vector<double> history_rsrp;
17     std::vector<double> history_timestamps;
18
19     //
20     std::vector<MapPoint> heatmap_points;
21 };

```

2.6.4 Результаты полевых испытаний

Система была протестирована в реальных условиях городской застройки. Маршрут тестирования включал различные типы местности: открытые пространства, плотную высотную застройку, подземные паркинги и тоннели. Результаты тестирования показали:

- Стабильную работу приложения в фоновом режиме в течение нескольких часов;
- Корректную привязку телеметрии к GPS-координатам с точностью до 5-10 метров;
- Успешную передачу данных через ZeroMQ без потерь при движении со скоростью до 60 км/ч;
- Наглядную визуализацию "мертвых зон" на тепловой карте.

ЗАКЛЮЧЕНИЕ

За прошедший учебный год в рамках цикла лабораторных работ был пройден путь от создания простейших мобильных экранных форм до разработки полноценного программно-аппаратного комплекса для радиочастотного мониторинга. В процессе работы были освоены и закреплены на практике следующие ключевые компетенции:

1. Проектирование распределенных систем

В ходе работы получены навыки связывания в единую экосистему компонентов, написанных на разных языках программирования и работающих на разных платформах. Взаимодействие между Android-клиентом (Kotlin) и Backend-сервером (C++) через высокопроизводительную шину сообщений ZeroMQ позволило на практике понять принципы построения отказоустойчивых сетевых архитектур. Освоены шаблоны обмена сообщениями (PUSH/PULL, PUB/SUB) и методы обеспечения гарантированной доставки данных.

2. Низкоуровневая работа с данными сотовых сетей

Был получен глубокий практический опыт работы с подсистемами операционной системы Android, в частности с `TelephonyManager` и `LocationManager`. Изучен физический смысл параметров радиосигнала (RSRP, RSRQ, RSSI, SINR, EARFCN, PCI) и реализованы алгоритмы их извлечения в реальном времени, что позволило превратить обычный смартфон в профессиональный измерительный прибор.

3. Высокопроизводительная визуализация на C++

Освоение библиотек `ImGui` и `ImPlot` позволило реализовать графический интерфейс, способный обрабатывать и отрисовывать большие массивы данных (тысячи точек на графиках и картах) без потери производительности. Изучена работа с графическим конвейером OpenGL: загрузка текстур, управление видеопамятью, отрисовка примитивов.

4. Работа с ГИС и картографией

Одним из самых сложных и интересных этапов стало внедрение карт OpenStreetMap. Был освоен математический аппарат проекции Меркатора, реализованы алгоритмы асинхронной загрузки тайлов через библиотеку `libcurl` и система их локального кеширования. Это позволило визуализировать результаты драйв-тестов на реальной местности без необходимости постоянного подключения к интернету.

5. Системное мышление и управление данными

Интеграция промышленной СУБД PostgreSQL и контейнеризация через Docker заложили фундамент для работы с большими данными. Были освоены навыки проектирования реляционных схем для хранения телеметрии (в том числе нормализация и создание индексов), написания параметризованных SQL-

запросов и обеспечения целостности данных при многопоточном доступе.

6. Анализ данных и машинное обучение

Реализация алгоритма пространственной интерполяции IDW для построения тепловых карт покрытия сети позволила на практике применить методы вычислительной геометрии и численных методов. Теперь система способна не только отображать единичные измерения, но и строить непрерывную картину покрытия на основе ограниченного набора точек.

7. Инженерная культура и рефакторинг

Разработка масштабного проекта приучила к дисциплине в использовании систем контроля версий (Git), написанию понятной документации в README и постоянному рефакторингу кода для разделения сущностей и повышения читаемости. Все этапы разработки фиксировались в отдельных ветках репозитория с последующим слиянием в основную ветку после успешного тестирования.

Перспективы развития проекта

Разработанный ПАК может быть расширен в следующих направлениях:

- Добавление поддержки 5G NR (New Radio) с новыми параметрами качества;
- Интеграция с картами Yandex и Google Maps;
- Создание веб-интерфейса для удаленного доступа к данным;
- Внедрение алгоритмов машинного обучения для предсказания качества сигнала;
- Разработка мобильного приложения для iOS.

В заключение можно с уверенностью сказать, что поставленные цели полностью достигнуты, а полученные знания и навыки будут полезны в дальнейшей профессиональной деятельности в области телекоммуникаций и разработки программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ахпашев Р.В. Визуальное программирование: методические указания к лабораторным работам [Электронный ресурс]. — URL: <https://telecomdep.github.io> (дата обращения: 28.05.2026).
2. Документация библиотеки ZeroMQ. — URL: <https://zeromq.org/> (дата обращения: 28.05.2026).
3. Документация OpenStreetMap. — URL: <https://wiki.openstreetmap.org/> (дата обращения: 28.05.2026).
4. Документация PostgreSQL (libpq). — URL: <https://www.postgresql.org/docs/current/> (дата обращения: 28.05.2026).
5. Документация ImGui. — URL: <https://github.com/ocornut/imgui> (дата обращения: 28.05.2026).
6. Документация ImPlot. — URL: <https://github.com/epezent/implot> (дата обращения: 28.05.2026).
7. Shepard D. A two-dimensional interpolation function for irregularly-spaced data // Proceedings of the 1968 23rd ACM National Conference. — 1968. — С. 517–524.
8. 3GPP TS 36.133 — Requirements for support of radio resource management (LTE).
9. Stroustrup B. The C++ Programming Language. — 4th ed. — Addison-Wesley, 2013. — 1347 с.
10. Tanenbaum E., Bos H. Modern Operating Systems. — 5th ed. — Pearson, 2022. — 1152 с.